

Orchestrator Field Manual

A complete, line-by-line course on the agent orchestration code built for Epic 3 — tools, memory, LangGraph, and streaming — written in plain B2 English, with a built-in dictionary for every hard word.

10 modules • 1 diagram • full glossary at the end

How to use this document. Every hard word is marked like this the first time it appears in a module — check the small **Vocabulary** box at the end of that module. A full alphabetical dictionary of every hard word is also in the last section. Read the modules in order — each one builds on the last.

CONTENTS

01 The Big Picture

02 The Tool Contract — tools/base.py

03 The First Real Tool — tools/search_knowledge.py

04 Talking to the Model — llm_client.py

05 Memory Without Writing Memory — memory.py

06 The Heart of the System — orchestrator.py

07 The HTTP Door — schemas.py & router.py

08 One Question, Start to Finish

09 Testing an Agent Without Calling a Real LLM

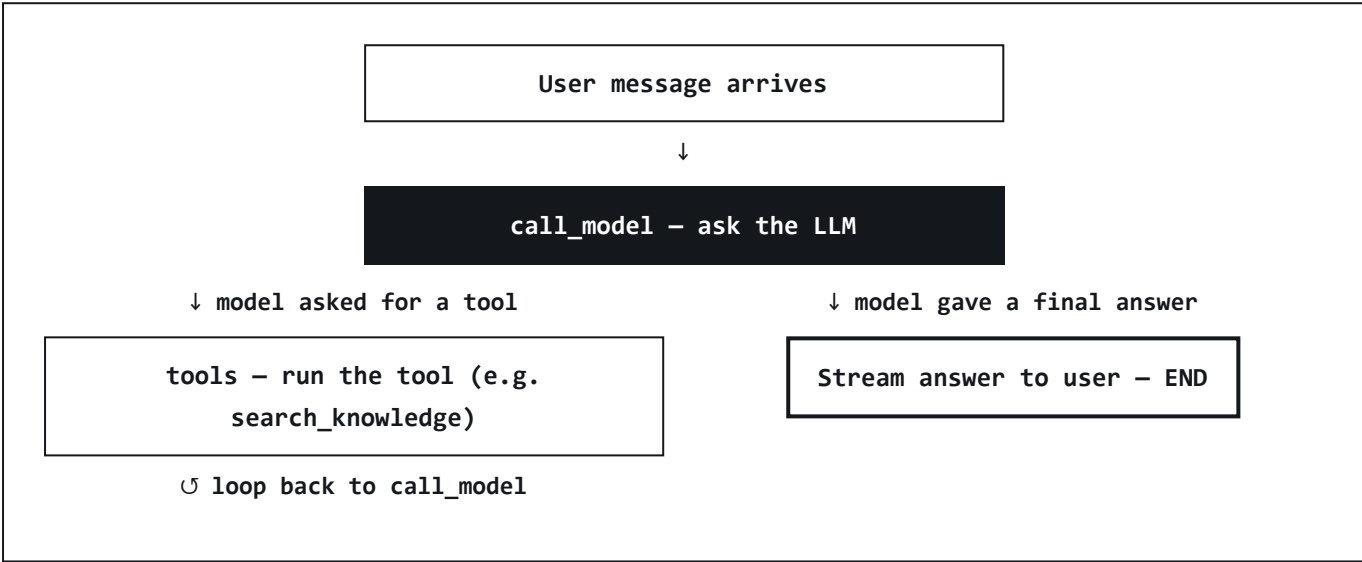
10 What Comes Next — Epic 4

§ Full Glossary

01 The Big Picture

Epic 3 adds one new capability to MindOps: a conversation endpoint, `POST /agent/chat`, where a user asks a question in plain language and an AI model answers — using the RAG knowledge base built in Epic 1 when it needs facts. The part of the code that makes this possible is called the **orchestrator**: the piece that decides, turn by turn, whether the model should answer directly or first call a **tool** (a small piece of code the model is allowed to run, such as "search the knowledge base").

The orchestrator is built with **LangGraph**, a library that models this decision loop as a **graph**: a small number of **nodes** (steps) connected by **edges** (arrows that say "go here next"). Our graph has exactly two nodes:



This is the whole idea. The model is asked a question; if it decides it needs information, it says so ("call `search_knowledge` with query X"); our code runs that tool and hands the result back; the model is asked again, now with that extra information; this repeats until the model is confident enough to give a real answer. A safety limit stops the loop if it never converges.

Seven files carry this out, and each one is small on purpose — every file has exactly one job:

FILE	JOB
agent/tools/base.py	Defines what "a tool" must look like, in the abstract.
agent/tools/search_knowledge.py	The one real tool: search the RAG knowledge base.
agent/llm_client.py	Builds the connection to the language model (Ollama).
agent/memory.py	Remembers earlier turns of a conversation.
agent/orchestrator.py	The graph itself — the decision loop.
agent/schemas.py	The shape of an incoming chat request.
agent/router.py	The HTTP endpoint that ties everything together and streams the answer.

VOCABULARY

orchestrator

The part of a program that controls the order in which other parts run. Here, it decides "ask the model" vs. "run a tool" vs. "stop".

tool

A small function the AI model is allowed to call, with a name and a description, so the model can decide when to use it.

node

One step in a graph — a single piece of work, like "call the model" or "run the tools".

edge

An arrow in a graph that connects one node to the next.

02 The Tool Contract

app/agent/tools/base.py

Before writing any real tool, we need a rule that every tool must follow — an **interface**. An interface is a promise: "any class that claims to be a Tool must have these things", without saying yet how each tool works inside. Here is the whole file:

```
from abc import ABC, abstractmethod
from typing import ClassVar

from pydantic import BaseModel

class Tool(ABC):
    """Port implémenté par chaque outil exposé au LLM via function calling."""

    name: ClassVar[str]
    description: ClassVar[str]
    args_schema: ClassVar[type[BaseModel]]

    @abstractmethod
    async def execute(self, **kwargs: object) -> str:
        """Exécute l'outil et renvoie un résultat textuel à réinjecter au LLM."""
```

ABC stands for **Abstract Base Class**. It is Python's way of saying "you can never create a bare **Tool** object — you must write a subclass that fills in the missing piece." The missing piece is marked with **@abstractmethod**: any subclass that forgets to write **execute()** will crash immediately when Python tries to load it, not later at 2 a.m. in production. That is the entire point of an abstract method — it turns a silent mistake into a loud, early one.

The three lines above **execute** are **class variables** (marked with **ClassVar**), meaning every tool must set a **name**, a **description**, and an **args_schema** once, as plain attributes — not as something computed per call. **args_schema** is a **Pydantic** model: a Python class that describes, and automatically checks, the shape of the arguments a tool accepts. This one schema is used for two different jobs at once — validating the arguments in Python, *and* generating the JSON description that tells the language model "here is what this tool expects." One definition, no duplication.

WHY AN INTERFACE AT ALL?

The orchestrator (module 06) never needs to know anything about a specific tool — not **search_knowledge**, not any future tool. It only needs to know that every tool has a **name**, a **description**, an **args_schema**, and an **execute()** method it can **await**. This is what lets us add a second, third, or tenth tool later without ever touching **orchestrator.py**.

VOCABULARY

abstract class

A class that cannot be used directly — it only exists to be extended by more specific subclasses.

abstract method

A method declared but not written in the abstract class; every subclass is forced to write it.

subclass

A new class built "on top of" an existing one, inheriting its shape and adding or filling in details.

schema

A precise description of the expected shape of some data (which fields exist, what type each one is).

validate

To check that a piece of data actually matches its expected schema before using it.

03 The First Real Tool

```
app/agent/tools/search_knowledge.py
```

This is the first class that actually fills in the promise from module 02. It wraps the RAG search function already built in Epic 1 (`rag/retriever.py`) — the orchestrator does not reinvent search, it reuses it.

```
from pydantic import BaseModel, Field
from sqlalchemy.ext.asyncio import AsyncSession

from app.agent.tools.base import Tool
from app.rag.embeddings.base import EmbeddingProvider
from app.rag.retriever import search as search_chunks

class SearchKnowledgeArgs(BaseModel):
    query: str = Field(
        description="La question ou le sujet à rechercher dans la base de connaissances"
    )

class SearchKnowledgeTool(Tool):
    """Outil de lecture seule : cherche dans la base de connaissances RAG (Epic 1)."""

    name = "search_knowledge"
    description = (
        "Cherche dans la base de connaissances les fragments les plus pertinents pour répondre "
        "à une question factuelle. À utiliser avant de répondre sur un sujet du domaine."
    )
    args_schema = SearchKnowledgeArgs

    def __init__(self, db: AsyncSession, provider: EmbeddingProvider, top_k: int = 5):
        self._db = db
        self._provider = provider
        self._top_k = top_k

    async def execute(self, *, query: str) -> str:
        results = await search_chunks(self._db, query, self._provider, top_k=self._top_k)
        if not results:
            return "Aucun fragment pertinent trouvé dans la base de connaissances."
        return "\n\n".join(
            f"[source: document#{chunk.document_id}] {chunk.text}" for chunk, _score in results
        )
```

Notice that `SearchKnowledgeTool` is not built once and reused forever — the `__init__` takes a database session and an embedding provider as arguments. That is because a database connection belongs to *one* incoming HTTP request; it cannot be shared safely between two users chatting at the same time. So a fresh `SearchKnowledgeTool` is created for every single chat request (you will see this in module 07). This is different from a tool that has no per-request state — such a tool could be built once and reused forever, which is exactly what module 04 does for the language model connection.

The `description` string is not a comment for humans — it is sent to the language model as part of the tool definition. The model reads it to decide *when* to call this tool. A vague description produces a model that calls the

tool at the wrong times, or never; write it the way you would explain the tool to a new colleague.

VOCABULARY

session (database)

A temporary connection to the database, open for the life of one request, then closed.

embedding

A list of numbers that represents the meaning of a piece of text, used to compare how similar two texts are.

per-request state

Information that must be created fresh for each incoming request, and never reused across different requests.

04 Talking to the Model

app/agent/llm_client.py

```
from functools import lru_cache

from langchain_core.language_models import BaseChatModel
from langchain_ollama import ChatOllama

from app.config import get_settings

@lru_cache
def get_llm_client() -> BaseChatModel:
    settings = get_settings()
    if settings.llm_provider != "ollama":
        raise ValueError(f"Fournisseur LLM non supporté : {settings.llm_provider}")
    return ChatOllama(model=settings.llm_model, base_url=settings.ollama_base_url, temperature=0)
```

This file is deliberately thin. Its only job is to build one object — a `ChatOllama` instance, which is LangChain's standard wrapper around a model running through Ollama on your own machine. `@lru_cache` is a decorator that turns this function into a singleton factory: the very first call actually builds the connection object; every call after that returns the exact same object instantly, instead of rebuilding it. This matters because opening a fresh connection on every single chat message would be wasteful.

`temperature=0` tells the model to be as deterministic as possible — pick its most likely next word every time, rather than adding randomness. For an agent that needs to reliably decide "should I call a tool or not," predictability is worth more than creative variety.

WHY HIDE THIS BEHIND A FUNCTION AT ALL?

Nothing in `orchestrator.py` or `router.py` imports `ChatOllama` directly. They only ever ask for "the current LLM client" through this one function — which FastAPI's dependency system calls for us (see module 07). In a test, we simply replace `get_llm_client` with a fake object that returns scripted answers, and nothing else in the code needs to change. This one layer of indirection is what makes the whole system testable without a real model running.

VOCABULARY

decorator

A special marker (written with @) placed above a function that changes how that function behaves, without rewriting its body.

singleton

An object that is only ever created once, then reused everywhere it is needed.

deterministic

Always producing the same output for the same input — the opposite of random.

05 Memory Without Writing Memory

app/agent/memory.py

```
from langgraph.checkpoint.memory import MemorySaver

# Historique de conversation en mémoire, indexé par thread_id (= conversation_id).
# Remplaçable par un checkpointer Postgres (langgraph-checkpoint-postgres) sans toucher
# à l'orchestrateur : c'est tout l'intérêt du checkpointer LangGraph.
checkpointer = MemorySaver()

def config_for(conversation_id: str) -> dict:
    return {"configurable": {"thread_id": conversation_id}}
```

A first version of this file could have been a plain Python dictionary mapping a conversation ID to a list of past messages, written by hand. We did not write that. LangGraph already has a built-in concept for this, called a **checkpointer**: every time the graph finishes a step, it automatically saves the full state (the whole list of messages so far) under a key called a **thread ID**. The next time you call the graph with the same thread ID, LangGraph loads that saved state back in before doing anything else.

MemorySaver is the simplest possible checkpointer — it keeps everything in the process's own memory, so it disappears if the server restarts. That is fine for a proof of concept. The important design decision is that **nothing else in the codebase needs to know this**: if this line ever becomes **PostgresSaver(...)** instead, so conversations survive a restart, no other file changes. The "conversation_id" the user sends us is simply reused as LangGraph's "thread_id" — one concept wearing two names.

VOCABULARY

checkpoint

A saved snapshot of a program's state at one moment, that can be loaded back later.

thread (in this context)

Not a computing "thread" — here it simply means "one ongoing conversation," identified by an ID.

persistence

The ability of data to survive after the program that created it stops running.

06 The Heart of the System

`app/agent/orchestrator.py`

Everything so far exists to support this one file. Read it slowly — every later module in this course refers back to it.

```

from collections.abc import AsyncIterator, Sequence

from langchain_core.language_models import BaseChatModel
from langchain_core.messages import AIMessage, HumanMessage, SystemMessage, ToolMessage
from langgraph.checkpoint.base import BaseCheckpointSaver
from langgraph.errors import GraphRecursionError
from langgraph.graph import END, MessagesState, StateGraph

from app.agent.tools.base import Tool

SYSTEM_PROMPT = (
    "Tu es un assistant qui répond aux questions en t'appuyant sur la base de connaissances "
    "via l'outil search_knowledge quand c'est pertinent. Cite les sources utilisées."
)

MAX_ITERATIONS = 5
RECURSION_LIMIT = MAX_ITERATIONS * 2

def build_graph(llm: BaseChatModel, tools: Sequence[Tool], checkpointer: BaseCheckpointSaver):
    tools_by_name = {tool.name: tool for tool in tools}
    tool_defs = [
        {
            "name": tool.name,
            "description": tool.description,
            "parameters": tool.args_schema.model_json_schema(),
        }
        for tool in tools
    ]
    llm_with_tools = llm.bind_tools(tool_defs) if tool_defs else llm

    async def call_model(state: MessagesState) -> dict:
        messages = state["messages"]
        if not messages or not isinstance(messages[0], SystemMessage):
            messages = [SystemMessage(content=SYSTEM_PROMPT), *messages]
        response = await llm_with_tools.ainvoke(messages)
        return {"messages": [response]}

    async def call_tools(state: MessagesState) -> dict:
        last_message = state["messages"][-1]
        results = []
        for call in last_message.tool_calls:
            tool = tools_by_name.get(call["name"])
            if tool is None:
                content = f"Outil inconnu : {call['name']}"
            else:
                content = await tool.execute(**call["args"])
            results.append(ToolMessage(content=content, tool_call_id=call["id"]))
        return {"messages": results}

    def route_after_model(state: MessagesState) -> str:
        last_message = state["messages"][-1]
        if isinstance(last_message, AIMessage) and last_message.tool_calls:
            return "tools"
        return END

    graph = StateGraph(MessagesState)
    graph.add_node("call_model", call_model)

```

```

graph.add_node("tools", call_tools)
graph.set_entry_point("call_model")
graph.add_conditional_edges("call_model", route_after_model, {"tools": "tools", END: END})
graph.add_edge("tools", "call_model")

return graph.compile(checkpointer=checkpointer)

async def stream_chat(app, *, conversation_id: str, user_message: str) -> AsyncIterator[str]:
    config = {"configurable": {"thread_id": conversation_id, "recursion_limit": RECURSION_LIMIT}}
    inputs = {"messages": [HumanMessage(content=user_message)]}
    try:
        async for chunk, metadata in app.astream(inputs, config=config, stream_mode="messages"):
            if metadata.get("langgraph_node") == "call_model" and chunk.content:
                yield chunk.content
    except GraphRecursionError:
        yield "\n\n[Erreur : nombre maximum d'itérations atteint sans réponse finale.]"

```

The state

`MessagesState` is a ready-made LangGraph state shape: a dictionary with one key, `"messages"`, holding the whole conversation as a growing list. Every node receives the current state and returns a small dictionary describing what changed — never the whole state rewritten by hand. LangGraph then merges that small change into the real state using a reducer: a function whose only job is "combine the old state with this new piece." For messages specifically, the reducer's rule is *"append this new message to the list, unless a message with the exact same ID is already there — then replace it instead."* Keep that rule in mind; module 09 tells the story of a bug caused by forgetting it.

The two nodes

`call_model` sends the whole message history to the language model and gets one response back — either plain text, or a request to call one or more tools (never both, for the models we use here). `call_tools` does the opposite job: it looks at the last message, finds every tool call the model asked for, actually runs each one by calling `tool.execute(**call["args"])`, and wraps every result in a `ToolMessage` — a special message type that *must* carry the same `tool_call_id` the model used to ask for it, so the model can match the answer to its own question.

The routing decision

`route_after_model` is the only place the loop actually "decides" anything. If the model's last message contains tool calls, go run them; otherwise, the conversation is over for this turn — go to `END`. This function is wired into the graph with `add_conditional_edges`, which is LangGraph's way of saying "after this node, ask this function which node comes next."

The safety limit

An agent that keeps calling tools forever, never satisfied, is a real failure mode — a badly-behaved model, or a tool that always returns something the model doesn't trust. `recursion_limit` caps how many steps the whole graph is allowed to take in total before LangGraph raises a `GraphRecursionError` on its own, without our code writing any counter by hand. We catch that specific error in `stream_chat` and turn it into a normal message for the user instead of an ugly server crash.

Streaming

`app.astream(..., stream_mode="messages")` is what makes the answer appear word by word instead of all at once at the end. It yields small pieces (chunks) of whatever a node is producing, tagged with metadata saying which node produced them. We only forward the chunks coming from `call_model` — the tool-calling messages are not meant for the user's eyes.

VOCABULARY

state (in a graph)

All the information the graph is currently carrying — here, the full message history of one conversation.

reducer

A function that takes the old state and a proposed change, and produces the new, combined state.

conditional edge

A connection between two graph nodes that is only taken when a certain condition is true.

recursion

A process that repeats itself, sometimes calling back into its own steps — here, the loop of model → tool → model.

chunk

One small piece of a bigger response, sent as soon as it is ready instead of waiting for the whole thing.

07 The HTTP Door

app/agent/schemas.py & app/agent/router.py

The schema is short and simply describes what a chat request must contain:

```
from pydantic import BaseModel, Field

class ChatRequest(BaseModel):
    conversation_id: str | None = Field(default=None, examples=["conv-1"])
    message: str = Field(min_length=1, examples=["Pourquoi le paiement échoue ?"])
```

The router is where every piece from modules 02–06 is finally wired together, for one single incoming HTTP request:

```

import json
import uuid
from collections.abc import AsyncIterator

from fastapi import APIRouter, Depends
from fastapi.responses import StreamingResponse
from langchain_core.language_models import BaseChatModel
from sqlalchemy.ext.asyncio import AsyncSession

from app.agent.llm_client import get_llm_client
from app.agent.memory import checkpointer
from app.agent.orchestrator import build_graph, stream_chat
from app.agent.schemas import ChatRequest
from app.agent.tools.search_knowledge import SearchKnowledgeTool
from app.core.database import get_db
from app.rag.embeddings import EmbeddingProvider, get_embedding_provider

router = APIRouter()

async def _sse_events(app, *, conversation_id: str, message: str) -> AsyncIterator[str]:
    yield f"event: start\ndata: {json.dumps({'conversation_id': conversation_id})}\n\n"
    async for token in stream_chat(app, conversation_id=conversation_id, user_message=message):
        yield f"event: delta\ndata: {json.dumps({'text': token})}\n\n"
    yield "event: done\ndata: {}\n\n"

@router.post("/chat")
async def chat(
    payload: ChatRequest,
    db: AsyncSession = Depends(get_db),
    provider: EmbeddingProvider = Depends(get_embedding_provider),
    llm: BaseChatModel = Depends(get_llm_client),
) -> StreamingResponse:
    conversation_id = payload.conversation_id or str(uuid.uuid4())
    tools = [SearchKnowledgeTool(db=db, provider=provider)]
    app = build_graph(llm, tools, checkpointer)

    return StreamingResponse(
        _sse_events(app, conversation_id=conversation_id, message=payload.message),
        media_type="text/event-stream",
    )

```

Look closely at the three `Depends(...)` arguments. This is **dependency injection**: instead of the `chat` function building its own database session or its own LLM client, it simply declares "I need one of these" and FastAPI provides it automatically, right before the function runs. The huge benefit shows up in testing (module 09): a test can tell FastAPI "whenever anyone asks for `get_llm_client`, give them my fake instead" — and nothing in `router.py` or `orchestrator.py` needs to change or even know.

Every request builds a brand-new `SearchKnowledgeTool` (bound to *this* request's database session) and a brand-new graph via `build_graph(...)` — but the `checkpointer` is imported as the same shared object every time, so conversation history still survives across separate requests, even though the graph object itself is rebuilt each time.

The response format is **Server-Sent Events** (SSE): a simple, one-directional streaming protocol built on top of plain HTTP, where the server writes small text blocks starting with `event:` and `data:`, separated by blank lines,

and the browser (or any HTTP client) reads them as they arrive instead of waiting for the connection to close.

VOCABULARY

dependency injection

A design pattern where a function declares what it needs, and an outside system provides it, instead of the function building it itself.

endpoint

One specific URL and HTTP method that a server responds to, such as POST /agent/chat.

Server-Sent Events (SSE)

A way for a server to push a stream of small text messages to a client over one open HTTP connection.

payload

The actual data carried inside a request or a message, as opposed to its headers or metadata.

08 One Question, Start to Finish

Here is exactly what happens when a user asks: *"Pourquoi le paiement échoue ?"* — with no prior conversation.

- 1 The client sends `POST /agent/chat` with `{"message": "Pourquoi le paiement échoue ?"}` and no `conversation_id`.
- 2 FastAPI validates the body against `ChatRequest`, then resolves the three `Depends(...)`: a database session, the embedding provider, and the (cached) Ollama client.
- 3 `chat()` generates a new UUID for `conversation_id`, builds one `SearchKnowledgeTool` bound to this request's database session, and calls `build_graph(...)` to compile a fresh graph wired to the shared `checkpointer`.
- 4 The response starts streaming immediately: the client receives `event: start` with the new conversation ID.
- 5 Inside `stream_chat`, the graph's `call_model` node runs first. It prepends the system prompt and sends the single human message to `llama3.1:8b` through Ollama.
- 6 The model decides it needs facts, and answers with a tool call: `search_knowledge(query="paiement échoue")`, carrying no visible text yet — so nothing is streamed to the user during this step.
- 7 `route_after_model` sees a tool call and sends the graph to the `tools` node.
- 8 `call_tools` runs `SearchKnowledgeTool.execute(query=...)`, which calls the RAG retriever from Epic 1, gets back the most relevant chunks, and wraps the formatted text in a `ToolMessage`.
- 9 The graph loops back to `call_model`, this time with the tool's answer included in the message history.
- 10 The model now writes a real answer, in plain text, citing the source — and this time, every word is streamed to the client as an `event: delta`, as soon as it is generated.
- 11 `route_after_model` sees no tool call this time, and the graph reaches `END`. The stream sends a final `event: done` and the HTTP connection closes.
- 12 The `MemorySaver` checkpointer has already stored every message from this exchange under the new `conversation_id` — so a follow-up question using the same ID starts with full context.

09 Testing an Agent Without Calling a Real LLM

A real language model is slow, costs money or GPU time, and gives a slightly different answer every time — none of which is acceptable in an automated test. So the tests never call Ollama. Instead, `tests/agent/fakes.py` defines a `ScriptedChatModel`: a fake that inherits from LangChain's own `BaseChatModel` base class, and simply replays a fixed list of pre-written responses, one per call, instead of thinking.

```
class ScriptedChatModel(BaseChatModel):
    responses: list[AIMessage]
    _call_count: int = PrivateAttr(default=0)

    def bind_tools(self, tools, **kwargs):
        return self # les outils declares ne changent rien au script

    def _generate(self, messages, stop=None, run_manager=None, **kwargs):
        index = min(self._call_count, len(self.responses) - 1)
        self._call_count += 1
        template = self.responses[index]
        message = AIMessage(content=template.content, tool_calls=template.tool_calls)
        return ChatResult(generations=[ChatGeneration(message=message)])
```

Because `get_llm_client` is only ever accessed through FastAPI's dependency injection (module 07), a test can override it with one line — `app.dependency_overrides[get_llm_client] = lambda: my_fake` — and the real router and the real orchestrator run completely unmodified against the fake. This is the payoff of every abstraction built in modules 02–04.

A REAL BUG WE HIT WHILE BUILDING THIS

The very first version of `ScriptedChatModel` reused the exact same `AIMessage` Python object every time the script ran out of new responses. A test that expected the loop to run forever (to check the recursion limit) instead stopped after a single tool call, silently, with no error.

The cause: LangGraph's message reducer (module 06) merges new messages by identity — it checks each message's `id` field. Returning the same object twice means returning the same `id` twice, so the reducer read it as "update this existing message in place," not "add a new message to the end." The message list stopped growing, the last message in the state was no longer the model's reply, and the routing check silently took the wrong path.

The fix was one line: build a brand-new `AIMessage` on every call, even when the content is identical, so it gets a fresh, unique `id`. This is a good lesson for any system built around a reducer that merges by identity: never assume "same content" means "same object" is safe to reuse.

VOCABULARY

fake / mock

A simplified stand-in for a real, complex, or slow component, used only during tests.

override (a dependency)

To temporarily replace what a piece of code receives, usually only for the length of one test.

identity (of an object)

Whether two variables point to the very same object in memory, rather than two separate objects that merely look equal.

10 What Comes Next — Epic 4

Right now, the only tool the agent can call is read-only: `search_knowledge` never changes anything, so it always runs immediately, with no human approval needed. Epic 4 introduces the first tool that *acts* in the world — sending an email — and with it, the need for a human to approve or reject sensitive actions before they run.

This is exactly where LangGraph's **interrupt** mechanism becomes relevant, as discussed separately: a node can be marked so the graph pauses right before it runs, hands control back to our code, and waits for an external decision — approve or reject — before resuming exactly where it left off. That would let the "gating" queue described in the backlog (Epic 4) reuse LangGraph's own pause-and-resume machinery, instead of a hand-written queue and polling system.

Full Glossary — every hard word, A to Z

abstract class

A class that cannot be created directly — it exists only to be extended by more specific subclasses that fill in its missing pieces.

abstract method

A method that is declared in an abstract class but has no real code yet; every subclass is forced to write its own version.

chunk

One small piece of a bigger response or file, handled or sent on its own instead of all at once.

checkpoint

A saved snapshot of a program's state at one moment in time, which can be loaded back later to continue from that point.

conditional edge

A connection between two steps of a graph that is only followed when a certain condition is true.

decorator

A marker written with @ above a function, which changes how that function behaves without rewriting its body.

dependency injection

A pattern where a piece of code declares what it needs, and an outside system supplies it automatically, instead of building it itself.

deterministic

Always producing the same output for the same input, with no randomness involved.

edge

In a graph, an arrow or connection that links one node to another.

embedding

A list of numbers that represents the meaning of a piece of text, allowing a computer to compare how similar two texts are.

endpoint

One specific web address and HTTP method that a server listens on and responds to.

fake / mock

A simplified stand-in used in tests, replacing a real component that would be too slow, costly, or unpredictable to use directly.

graph (in this context)

A structure made of a small number of steps (nodes) connected by arrows (edges), used here to model a decision loop.

identity (of an object)

Whether two variables refer to the exact same object in memory, not merely to two equal-looking values.

interface

A defined shape or contract that different pieces of code agree to follow, without needing to know each other's inner details.

interrupt (LangGraph)

A pause built into a graph, where execution stops before or after a node and waits for an outside decision before it can continue.

node

One single step of work inside a graph.

orchestrator

The part of a program that decides the order in which other parts of the system should run.

override (a dependency)

To temporarily replace what a piece of code receives, most often only for the duration of one test.

payload

The actual, useful data carried inside a request or message, separate from its headers or metadata.

per-request state

Information that must be freshly created for a single incoming request and never shared with any other request.

persistence

The ability of data to remain available even after the program that created it has stopped running.

recursion

A process that repeats by calling back into its own earlier steps.

reducer

A function that takes the current state and a proposed small change, and produces the new, combined state.

schema

A precise description of the expected shape of a piece of data — which fields it has, and what type each one must be.

session (database)

A temporary connection to a database, kept open only for the life of one request.

Server-Sent Events (SSE)

A simple protocol allowing a server to push a stream of small text messages to a client over one open connection.

singleton

An object that is created only once and then reused every time it is needed, instead of being rebuilt.

state (in a graph)

All the information a graph is currently carrying as it moves from one node to the next.

subclass

A new class built on top of an existing one, inheriting its shape and adding or completing details.

thread (LangGraph sense)

Here, simply "one ongoing conversation," identified by an ID — unrelated to a computing thread.

tool

A small function that an AI model is allowed to call, described by a name, a description, and an expected set of arguments.

validate

To check that a piece of data truly matches its expected schema before it is used.